CSE 211 — Theory of Computation

Regular Operations, Closure Properties and Regular Expressions

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Hakim Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — closure proofs, REs and conversions in every paper

Lesson 10 — Regular Operations and Closure under ∪, ∘, ∗ — definitions, NFA constructions, $\emptyset^* = \{\varepsilon\}$, the star-construction trap

Lesson 11 — More Closure: Complement, Intersection, Difference, Reverse — flip a DFA (not an NFA), product construction

Lesson 12 — Regular Expressions — syntax, semantics, writing, describing, simplifying, choosing

Lesson 13 — RE → NFA (Thompson's Construction) — base cases + plug-in constructions

Lesson 14 — DFA → RE (GNFA State Ripping) and Kleene's Theorem — GNFA rules, the rip formula, proofs

Quick Notes — Regular Languages — revision bullets + key diagrams

NOTE Slide numbers refer to the PDF pages of Lec05 - The Regulars.pdf (111 pages). Every regular expression in the solutions was compared by a program with the language definition on all strings up to length 10 (3–6 for larger alphabets); GNFA results and Thompson NFAs were generated and tested the same way. RE notation: ∪ union, ∗ star, $^+$ one-or-more, Σ = (0 ∪ 1) or (a ∪ b). Practice provenance: Actual = taken from a TF paper; Adapted = TF question with changed details; Constructed = new question in TF style.

Exam Map — What the TF Papers Ask

Session	Questions on regular operations, closure and REs
2023-24	Q1(c) NFAs for concatenation of {w | ≥ three 1s} with ∅, via the closure construction (10) · Q3(b) Star construction without a new start state — why bad; counterexample (13) · Q4(b) Which RE (A–D) gives strings with at most one occurrence of aa? Reasons (12)
2022-23	Q3(b) Swapping accept states of an NFA: complement? Example (12) · Q3(c) Why is the Kleene star of the empty set the empty string? (8) · Q4(a) DFA D for L(D1) ∪ L(D2): L(D1) in English; start state + two reachable states (10) · Q4(c) Which RE (A–C) represents the 2-state DFA? Explain each option (10)
2021-22	Q1(c) Why swapping accept states of a DFA gives the complement; does it work for NFAs? (9) · Q2(b) NFA for ((((00)∗(11)) ∪ 01)∗ by basic principles (10) · Q2(c) Finite or infinite: L1∗∘L2∗, L1∘(L1∪L2), L2∘L3 (L1 = {a,aa,aaa}, L2 = {a,ab,aab,aabb}, L3 = #b multiple of 3) (6) · Q3(a) Why is the Kleene star of the empty set the empty string? (4) · Q3(d) Which RE (A–D) gives strings with both 00 and 11? Justify each (12) · Q4(b) Union NFA: start state, accept states, transitions from the start state — with reasons (12)
2020 (Jan term)	Q2(c) RE for the strings NOT in the language of 2(a) (10) · Q3(a) Evaluate ∅∗∅∗∅∗∅∗, ∅∗∪{ε}, (∅∗∪1){0,1}, (11)∗∅(00)∗, (∅∗∪0){0,1,11} (10) · Q3(b) Explain each case of δ in the star construction N for L1∗ (20) · Q4(a) Describe the language of (1+ε)(00∗1)∗0∗ (10)
2019-20	Q2(b) Simplify ∅∗Ua∗Ub∗∪(a∪b)∗; ((a∗b∗)∗(b∗a∗)∗)∗; (a∗b)∗∪(b∗a)∗ (15) · Q3(a) NFA for ((a∗)a)∗ by basic principles (5) · Q3(c) Union NFA N from N1, N2: write δ and justify each part (15) · Q4(b) REs over {a,b}: #a divisible by 3; at most three a's; exactly one occurrence of aaa (15)
2017-18	Q2(a) DFA for Σ∗ − L from DFA D; why it works; when not; does it work for NFAs? (10) · Q2(c) NFA for (a ∪ b)∗aba using closure constructions (8) · Q4(b) REs: each of a,b,c occurs; a^nb^m n≥3 m odd; (n+m) odd; vwv with |v|=2; #a mod 3 = 0 (20)
2016-17	Q2(c) Why is the Kleene star of the empty set the empty string? (10) · Q3(b) REs: alternating 0s and 1s; ≥1 a and ≥1 b over {a,b,c}; every 00 before any 11 (15) · Q4(a) Hasty's star construction (no new start state, old start accepting) — when does it fail? (10) · Q4(b) Prove closure under concatenation (13)
2015-16	Q2(b) Prove the regular languages are closed under the regular operations (15) · Q3(a) DFA and RE: begins 1 ends 0; 3rd symbol 0; even 0s or exactly two 1s; not 11/111; ≥2 0s and ≤1 1 (18) · Q3(b) Example: swapping accept states of an NFA does not give the complement (5⅓)
2014-15	Q3(b) REs: decimal numbers divisible by 5 (no leading 0s); odd 0s or odd 1s; no 101 (10)
2013-14	Q5(a) Regular languages are closed under difference L1 − L2 (8) · Q5(b) Counter-example to (R1 ∪ R2)∗ = R1∗ ∪ R2∗ [mark not printed; 5 by section total] (–) · Q6(c) RE from the given NFA (q0, q1, q2) (8) · Q8(a) Steps to construct a DFA for L1 ∪ L2 from DFAs M1, M2 (7)
2012-13	Q5(c) Define regular language; prove closure under star and concatenation (10) · Q6(c) RE for strings with an even number of 1s (3) · Q7(b) Convert the given DFA (states 1, 2, 3) to a regular expression (7⅓) · Q8(b) Prove a set is regular iff it can be described by a regular expression (12⅓)
2011-12	Q5(b) Define regular language; prove closure under star and concatenation (12) · Q7(a) NFA for (01 ∪ 001 ∪ 010)∗ (8) · Q7(c) Regular expression for the given 3-state automaton (states 1, 2, 3) (7⅓)
2009-10	Q4 L1 = 1(1∪0)∗000, L2 = (1(01)∗0)∗: is each of 1000, 1000101010, 10000111000100000 in L1∘L2? (10)
2008-09	Q7(a) Prove: if L = L(A) for a DFA A then L = L(R) for some RE R (14) · Q7(b) REs: (i) ≥1 a and ≥1 c over {a,b,c}; (ii) no two consecutive 0s or no two consecutive 1s (5) · Q7(c) Convert (0+1)∗1(0+1) to an ε-NFA (4⅓)
2007-08	Q2(b) Languages of (0 ∪ 1)∗ and of (0Σ∗) ∪ (Σ∗1) over {0,1} (8) · Q4(c) REs describe the same languages as automata — then why do we need REs? (5⅓)
2006-07	Q1(b) Regular expression for strings of alternating 0s and 1s (8) · Q4(b) Describe the languages of 8 REs: 0∗10∗, Σ∗1Σ∗, 1∗(01+)∗, (ΣΣΣ)∗, 0Σ∗0∪1Σ∗1∪0∪1, 01∪10, (0∪ε)1∗, (01∪010)∗ (8)

EXAM FOCUS The richest block of Section A (~25 % of its marks). Most repeated: (1) **write / choose / simplify REs** — every paper; 'which option is correct' in 2023-24, 2022-23, 2021-22; (2) **closure constructions explained** — union δ (2019-20, 2021-22), star construction and its flawed variant (2023-24, Jan-2020, 2016-17), concatenation (2023-24, 2016-17, 2011-12, 2012-13); (3) **complement by flipping a DFA, and why it fails for an NFA** — 2022-23, 2021-22, 2017-18, 2015-16; (4) ∅∗ = {ε} — 2022-23, 2021-22, 2016-17, Jan-2020; (5) RE → NFA by basic principles and DFA → RE.

Lesson 10 — Regular Operations and Closure under ∪, ∘, *

Slides 1–37 (Lec05)

1. The Three Regular Operations (sl. 4–13)

$$A \cup B = \{ x \mid x \in A \text{ or } x \in B \}$$
$$A \circ B = \{ xy \mid x \in A \text{ and } y \in B \}$$
$$A^* = \{ x_1x_2\ldots x_k \mid k \geq 0 \text{ and each } x_i \in A \}$$

Example (sl. 8–11): A = {good, bad}, B = {boy, girl}: A ∪ B has 4 strings, A ∘ B = {goodboy, goodgirl, badboy, badgirl}; {ab, c}∗ = {ε, ab, c, abab, abc, cab, cc, …}. Operations on languages are operations on sets of strings, not on machines; closure proofs ask whether a machine for the result can be built (sl. 6).

EXAM-READY ANSWER Why is ∅∗ = {ε}? (2022-23 Q3(c), 2021-22 Q3(a), 2016-17 Q2(c)) By definition A∗ = {ε} ∪ A ∪ AA ∪ … is the set of concatenations of k ≥ 0 strings of A. Choosing k = 0 concatenates nothing, which gives the empty string; this needs no member of A, so ε ∈ A∗ for every A, even A = ∅. For k ≥ 1 we must pick a string from ∅ — impossible — so those terms are empty. Hence ∅∗ = {ε} ∪ ∅ ∪ ∅ ∪ … = {ε} (sl. 12–13). Analogy: $x^0 = 1$ even for x = 0; the empty product is the identity of concatenation, which is ε. Consequence: the star NFA must accept ε, which is why its new start state is accepting.

2. Regular Languages and Closure (sl. 14–18)

A language is **regular** if some DFA (equivalently, some NFA) recognises it. A class is **closed** under an operation if applying it to members gives a member (ℕ is closed under +, not under −). Theorem: regular languages are closed under ∪, ∘ and ∗; the proofs use NFAs because ε-moves make them simple (sl. 16–17).

3. Union (sl. 19–24)

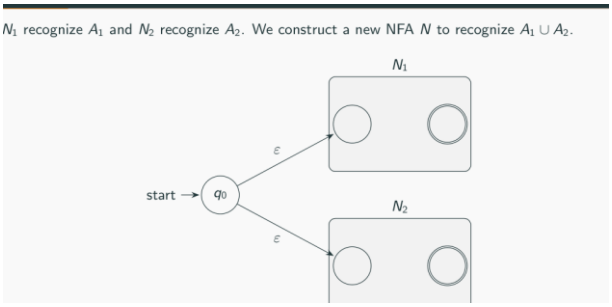


Figure 10.1 — Union: new start state with ε-moves to N1 and N2 (sl. 23)

$$N1 = (Q1, \Sigma, \delta1, q1, F1),\ N2 = (Q2, \Sigma, \delta2, q2, F2) \to N = (Q, \Sigma, \delta, q0, F) \text{ for } A1 \cup A2:$$

$$Q = \{q0\} \cup Q1 \cup Q2 \qquad q0 \text{ new start} \qquad F = F1 \cup F2$$

$$\delta(q, a) = \begin{cases} \delta1(q, a) & q \in Q1 \\ \delta2(q, a) & q \in Q2 \\ \{q1, q2\} & q = q0 \text{ and } a = \varepsilon \\ \emptyset & q = q0 \text{ and } a \neq \varepsilon \end{cases}$$

4. Concatenation (sl. 25–29)

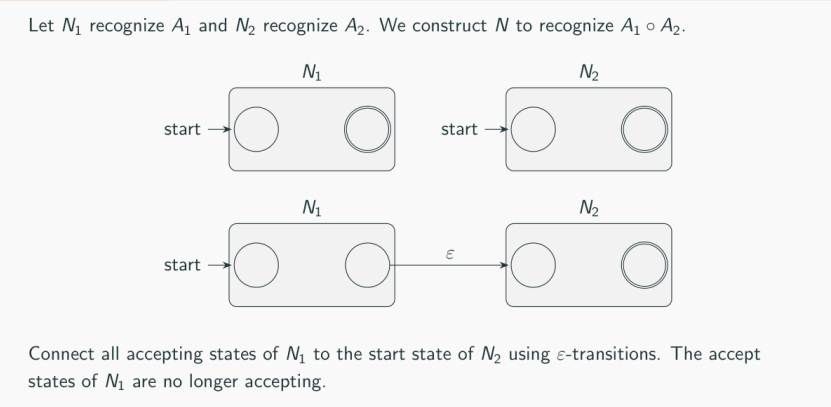


Figure 10.2 — Concatenation: ε from every accepting state of N1 to the start of N2 (sl. 27)

```
N for A1 ∘ A2:  Q = Q1 ∪ Q2,  start q1,  F = F2
              ⎧ δ1(q, a)            q ∈ Q1, q ∉ F1
  δ(q, a) =   ⎪ δ1(q, a)            q ∈ F1, a ≠ ε
              ⎪ δ1(q, a) ∪ {q2}     q ∈ F1, a = ε
              ⎩ δ2(q, a)            q ∈ Q2
```

At an accepting state of N1 the machine *guesses* the split point w = xy (sl. 29).

5. Star (sl. 30–33)

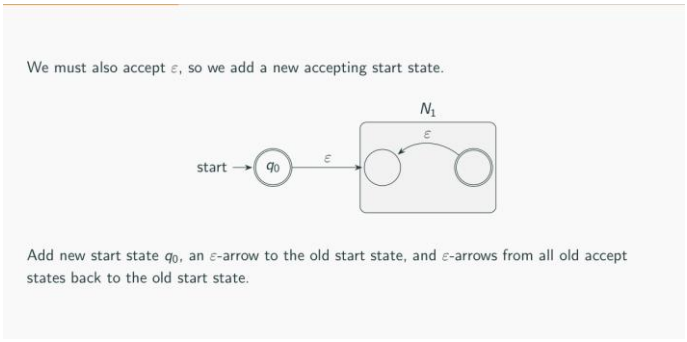


Figure 10.3 — Star: new accepting start state; ε from accepting states back to the old start (sl. 32)

```
N for A1∗:  Q = {q0} ∪ Q1,  start q0 (new),  F = {q0} ∪ F1
              ⎧ δ1(q, a)            q ∈ Q1, q ∉ F1
              ⎪ δ1(q, a)            q ∈ F1, a ≠ ε
  δ(q, a) =   ⎪ δ1(q, a) ∪ {q1}     q ∈ F1, a = ε      ← jump back: start the next piece
              ⎪ {q1}                q = q0, a = ε      ← enter N1
              ⎩ ∅                   q = q0, a ≠ ε      ← q0 reads nothing itself
```

TRAP Why a NEW start state? Making the old start q1 accepting (instead of adding q0) breaks the construction whenever N1 can return to q1 after reading a non-empty string that is not in A. Example (checked by program): N1 has q1 looping on a and q1 −b→ q2 (accepting); A = a∗b. A∗ = (a∗b)∗ — every non-empty member ends in b. With q1 accepting and q2 −ε→ q1, the machine accepts a, aa, ba — none in A∗. The fresh q0 has no incoming edges, so accepting it adds only ε.

6. Proof Template (sl. 34–37)

1. Start with machines for the input languages.
2. Describe the operation to simulate.
3. Build the new automaton (the construction is the heart).
4. Argue both directions: L(N) ⊆ L (accepts nothing extra) and L ⊆ L(N) (accepts everything required).

Exam Approach

1. Draw the picture (box N1, box N2, new states, ε arrows) and write Q, start, F, δ by cases.
2. Justify each case: what it keeps from N1/N2, what it adds, why nothing extra is accepted.
3. For 'finite or infinite' questions: ∘ of finite sets is finite; ∗ of a set containing a non-empty string is infinite.

Practice — Lesson 10

(a) Actual — 2021-22 Q4(b), 2019-20 Q3(c). (b) Actual — Jan-2020 Q3(b). (c) Actual — 2023-24 Q3(b), 2016-17 Q4(a). (d) Actual — 2023-24 Q1(c). (e) Actual — 2021-22 Q2(c). (f) Actual — 2015-16 Q2(b) / 2016-17 Q4(b) / 2011-12 Q5(b).

(a) N1 recognises L1 and N2 recognises L2; we build N for L1 ∪ L2. Explain how we get the start state, the accept states, and the transitions from the start state (cases, reasons, destinations). Write δ. (12–15)

(b) For N recognising L1∗ the transition function is defined by five cases (q ∈ Q1, q ∉ F1 / q ∈ F1, a ≠ ε / q ∈ F1, a = ε / q = q0, a = ε / q = q0, a ≠ ε). Explain each in plain English, stressing its purpose. (20)

(c) Someone suggests making the original start state accepting instead of adding a new start state for the star construction. Why is this a bad idea? Give a counterexample. (13)

(d) Draw NFAs for the concatenation of {w ∈ {0,1}∗ | w contains at least three 1s} with ∅, using the closure construction. (10)

(e) L1 = {a, aa, aaa}, L2 = {a, ab, aab, aabb}, L3 = {number of b's is a multiple of 3}: finite or infinite — L1∗ ∘ L2∗, L1 ∘ (L1 ∪ L2), L2 ∘ L3? (6)

(f) Prove that the class of regular languages is closed under the regular operations (concatenation, star). (15)

✓ Solution

(a) Start state: a new state q0 — neither N1's nor N2's start can serve, because the machine must be free to run either machine from the beginning. **Accept states:** F = F1 ∪ F2 — w ∈ L1 ∪ L2 iff N1 or N2 accepts it. **From q0:** on ε go to {q1, q2} (guess which machine will accept); on any real symbol a, δ(q0, a) = ∅ (q0 reads nothing; the first symbol is read inside N1 or N2). All other transitions are copied unchanged. δ: as in §3. Correctness: an accepting run of N starts q0 −ε→ qi and is then an accepting run of Ni, so L(N) ⊆ L1 ∪ L2; conversely choose the branch of the machine that accepts.

(b) Case 1 (q ∈ Q1, q ∉ F1): ordinary states behave exactly as in N1. Case 2 (q ∈ F1, a ≠ ε): accepting states keep their old moves, so a piece of A may continue to a longer piece. Case 3 (q ∈ F1, a = ε): additionally jump back to q1 — one piece of A is finished, start the next one (this gives A∘A, A∘A∘A, …). Case 4 (q = q0, a = ε): the new start enters N1 to read the first piece. Case 5 (q = q0, a ≠ ε): q0 itself reads nothing (∅), so q0 contributes only ε. q0 is accepting so that ε ∈ A∗ (k = 0 pieces).

(c) See the TRAP in §5: making q1 accepting accepts every string that brings N1 back to q1, even if it is not a concatenation of members of A. Counterexample A = a∗b: the flawed machine accepts a (and aa, ba), but a ∉ (a∗b)∗. The idea works only if q1 has no incoming transitions.

(d) N1 for 'at least three 1s': p0 −1→ p1 −1→ p2 −1→ p3, every state loops on 0, p3 loops on 0 and 1, F1 = {p3}. N2 for ∅: a single non-accepting start state r0 with no transitions. Concatenation: start p0, add p3 −ε→ r0, F = F2 = ∅. The result has no accepting state, so it recognises ∅ — as it must: L ∘ ∅ = ∅ for every L, because a string xy needs some y ∈ ∅.

```
p0 (start) –1→ p1 –1→ p2 –1→ p3;  p0,p1,p2 loop on 0;  p3 loops on 0,1;  p3 –ε→ r0;  r0: no
moves
accepting states: none  ⇒  L(N) = ∅
```

(e) L1∗ ∘ L2∗: **infinite** (L1∗ already contains a, aa, aaa, aaaa, …). L1 ∘ (L1 ∪ L2): **finite** — L1 ∪ L2 has 6 strings, so at most 3 × 6 = 18 concatenations. L2 ∘ L3: **infinite** — L3 is infinite (ε, bbb, bbbbbb, …) and L2 is non-empty, so a·b^(3k) gives infinitely many strings.

(f) Union: §3. Concatenation: §4 — if w = xy with x ∈ A1, y ∈ A2, N runs N1 on x, takes the ε-move from an accepting state of N1 to q2, and runs N2 on y; conversely every accepting run of N must cross from F1 to q2 exactly once (the only edges into Q2), splitting w into x ∈ A1 and y ∈ A2. Star: §5 — w = x1…xk with xi ∈ A is accepted by entering N1, running each xi and jumping back by ε; ε is accepted by q0; conversely each jump back marks the end of a piece in A. Since NFAs recognise exactly the regular languages, the class is closed.

Lesson 11 — More Closure: Complement, Intersection, Difference, Reverse

Slides 38–60 (Lec05)

1. Complement (sl. 38–43)

Ā = Σ∗ − A. If M = (Q, Σ, δ, q0, F) is a DFA for A, then M′ = (Q, Σ, δ, q0, Q − F) recognises Ā: every input follows exactly one complete path, which ends in F iff it does not end in Q − F. The DFA must be **complete** — add a dead state first, otherwise a string that 'gets stuck' would be rejected by both machines (sl. 42).

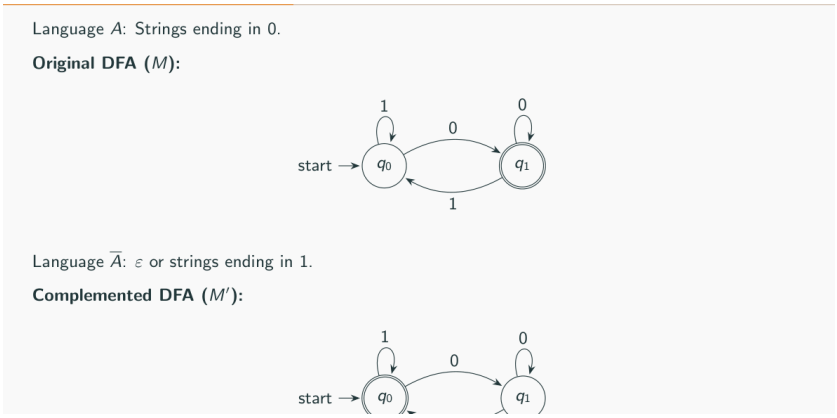


Figure 11.1 — Complementing a DFA: strings ending in 0 → ε or strings ending in 1 (sl. 43)

EXAM-READY ANSWER Why flipping fails for an NFA (2022-23 Q3(b), 2021-22 Q1(c), 2017-18 Q2(a), 2015-16 Q3(b)): an NFA accepts if SOME branch ends in an accepting state. After flipping, it accepts if some branch ends in a (formerly) non-accepting state. A string can have one branch of each kind, so it is accepted by both machines. Example over {a}: q0 (start, accepting) loops on a and also q0 −a→ q1 (non-accepting, no moves). L(N) = a∗, so the complement is ∅. Flipped: q0 non-accepting, q1 accepting: it accepts a, aa, … (branch that jumps to q1 on the last a) — not ∅. Strings rejected because every branch dies are also rejected after flipping. Correct method: convert the NFA to a DFA (subset construction), complete it, then flip.

2. Intersection: Product Construction and De Morgan (sl. 44–50)

Q = Q1 × Q2, q0 = (q1, q2), δ((r1, r2), a) = (δ1(r1, a), δ2(r2, a))
A ∩ B: F = F1 × F2 A ∪ B: F = (F1 × Q2) ∪ (Q1 × F2)

A state (r, s) means 'after the same prefix, M1 is in r and M2 is in s' — running two programs side by side (sl. 48). Algebraic proof: A ∩ B = complement of (Ā ∪ B̄) — complement, union, complement (sl. 49–50).

3. Difference and Reverse (sl. 51–57)

A − B = A ∩ B̄ (sl. 51–52). Reverse: reverse every arrow of a DFA for A, make the old start the only accepting state, and add a new start with ε-moves to all old accepting states — an NFA for A^R (reversal can create nondeterminism) (sl. 53–57).

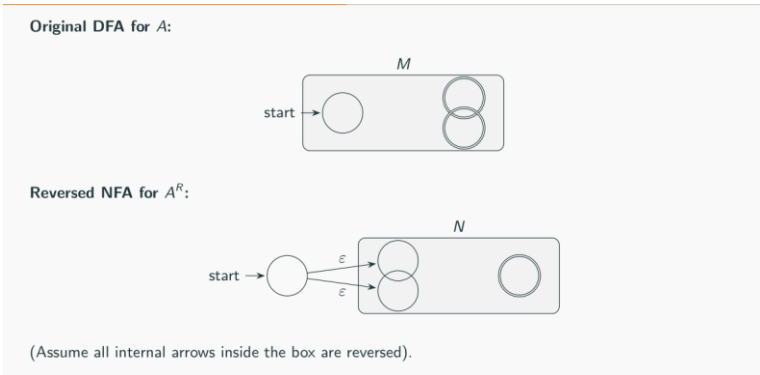


Figure 11.2 — Reversing a DFA gives an NFA (sl. 57)

4. Closure as a Proof Shortcut (sl. 58–60)

'Contains 101 but does not end in 11' = L1 − L2 with both regular ⇒ regular, with no diagram. Likewise 'contains 101 and has even length' = A ∩ B.

Exam Approach

1. Complement: make the DFA complete, flip F. For NFAs: convert first.
2. Intersection / union of DFAs: product states, list only the reachable pairs, choose F by AND / OR.
3. Closure-based regularity: express L with ∪, ∩, −, complement, reverse of known regular languages.

Practice — Lesson 11

(a) Actual — 2017-18 Q2(a) / 2021-22 Q1(c). (b) Actual — 2022-23 Q4(a). (c) Actual — 2013-14 Q5(a). (d) Actual — 2013-14 Q8(a). (e) Actual — 2013-14 Q8(c).

(a) Given a DFA D for L over Σ, how can you readily find the DFA for $\Sigma^* - L$? Why does it work? Is there any scenario in which it will not work? Does it work for NFAs? (9–10)

(b) D1: s0 (start, accepting) −0,1→ s1, s1 loops on 0,1. D2: q1 (start) loops on 0, q1 −1→ q2, q2 loops on 1, q2 −0→ q3, q3 −0,1→ q2; q2, q3 accepting. We build D for L(D1) ∪ L(D2) from basic principles. (i) What is L(D1)? (ii) Draw the start state of D and two states reachable from it. (10)

(c) Prove that regular languages are closed under L1 − L2 = {w | w ∈ L1 but w ∉ L2}. (8)

(d) M1, M2 are DFAs for L1, L2. Describe the steps to construct a DFA for L1 ∪ L2. (7)

(e) Show a counter-example to disprove: every subset of a regular language is also regular. (6)

✓ Solution

(a) Swap accepting and non-accepting states: M′ = (Q, Σ, δ, q0, Q − F). It works because a DFA has exactly one run on every input, ending in exactly one state, which is in F or in Q − F. It fails if the DFA is not complete (missing transitions: stuck strings are rejected by both) — add a dead state first; and the alphabet must be the full Σ. It does not work for NFAs — see the EXAM-READY ANSWER in §1 and its example.

(b) (i) D1 accepts only the empty string: L(D1) = {ε}. (ii) Product: start (s0, q1); on 0 → (s1, q1); on 1 → (s1, q2). (s0, q1) is accepting (s0 ∈ F1); (s1, q2) is accepting (q2 ∈ F2); (s1, q1) is not. From then on s1 never changes, so D behaves like D2: L(D) = {ε} ∪ L(D2), where L(D2) = strings with at least one 1 and an even number of 0s after the last 1.

```
(s0,q1) start,* -0→ (s1,q1)        (s1,q1) -0→ (s1,q1), -1→ (s1,q2)
        -1→ (s1,q2)*        (s1,q2) -1→ (s1,q2), -0→ (s1,q3)*
```

(c) L1 − L2 = L1 ∩ (complement of L2). If L2 is regular its complement is regular (flip a complete DFA); the intersection of two regular languages is regular (product construction with F = F1 × (Q2 − F2)). Directly: one product DFA with accepting pairs (r, s), r ∈ F1, s ∉ F2.

(d) 1. States Q1 × Q2. 2. Start ($q0^1$, $q0^2$). 3. δ((r, s), a) = (δ1(r, a), δ2(s, a)). 4. Accept (r, s) if r ∈ F1 **or** s ∈ F2. 5. Keep only pairs reachable from the start. Both DFAs must be complete over the same Σ.

(e) $\Sigma^* = \{0,1\}^*$ is regular, but its subset {0^n1^n | n ≥ 0} is not regular (pumping lemma, Lesson 16). So regular languages are not closed under taking subsets. (Every language over Σ is a subset of the regular language Σ^*.)

Lesson 12 — Regular Expressions

Slides 61–84 (Lec05)

1. Definition, Semantics, Precedence (sl. 61–68)

R is a regular expression if R is a (a ∈ Σ), ε, ∅, (R1 ∪ R2), (R1 ∘ R2) or (R1∗), where R1, R2 are REs (sl. 64). L(a) = {a}, L(ε) = {ε}, L(∅) = ∅, L(R1 ∪ R2) = L(R1) ∪ L(R2), L(R1R2) = L(R1)L(R2), L(R1∗) = L(R1)∗ (sl. 65). Shorthands: Σ, R^+ = RR∗, R? = R ∪ ε, R^k, [0-9] (sl. 66). Precedence: star, then concatenation, then union: 0 ∪ 10∗ = 0 ∪ (1(0∗)) (sl. 68).

2. Examples (sl. 69–84)

RE	Language
0∗10∗	exactly one 1
Σ∗1Σ∗	at least one 1
1∗∅	∅ (anything ∘ ∅ = ∅)
∅∗	{ε}
Σ∗1Σ∗1Σ∗	at least two 1s
(0 ∪ 1)∗00	ends in 00
(ΣΣ)∗	even length
(0 ∪ 1(01∗0)∗1)∗	binary multiples of 3 (and ε) — Lesson 14

MEMORIZE Identities (sl. 83–84): R ∪ ∅ = R · Rε = R · R ∪ R = R · (R∗)∗ = R∗ · RØ = ∅ · ∅∗ = {ε}. Also (R ∪ S)∗ = (R∗S∗)∗, and (R1 ∪ R2)∗ ≠ R1∗ ∪ R2∗ in general.

REs are not unique (sl. 104): (0 ∪ 1)∗1, 0∗1(1 ∪ 00∗1)∗ and Σ∗1 all describe 'ends in 1'.

3. How to Write an RE

1. Find the structure: blocks (a+b+), a mandatory part with free parts around it (Σ∗ x Σ∗), counts mod k (b∗(ab∗ab∗ab∗)∗), or 'forbidden pattern' (build from allowed blocks).
2. For 'A and B' (e.g. contains a and c) list the orders and take the union.
3. For 'not P' it is often easier to describe the allowed blocks, or to build a DFA and convert (Lesson 14).
4. Test: two members, two non-members, and ε.

EXAM-READY ANSWER Why do we need REs if automata describe the same languages? (2007-08 Q4(c)) REs are a compact, algebraic, declarative notation: easy to write and read in one line, easy to type into tools (grep, lexers, search/replace, input validation), and easy to combine and transform algebraically. Automata are the implementation; REs are the specification. Kleene's theorem lets us convert mechanically between them.

Exam Approach — Choosing Among RE Options

1. For each option find a shortest string that it wrongly accepts (false positive) or wrongly rejects (false negative). One counterexample per wrong option is enough.
2. For the correct option argue both directions in one sentence each.
3. Careful with overlapping occurrences: aaa contains aa twice.

Practice — Lesson 12 — writing REs

(a) Actual — 2019-20 Q4(b). (b) Actual — 2017-18 Q4(b). (c) Actual — 2016-17 Q3(b). (d) Actual — 2014-15 Q3(b). (e) Actual — 2008-09 Q7(b), 2012-13 Q6(c), 2006-07 Q1(b), Jan-2020 Q2(c).

(a) Over {a, b}: (i) number of a's divisible by three; (ii) no more than three a's; (iii) exactly one occurrence of the substring aaa. (15)

(b) (i) Σ = {a,b,c}, each symbol occurs at least once; (ii) {a^n b^m | n ≥ 3, m odd}; (iii) {a^n b^m | n + m odd}; (iv) {vwv | v, w ∈ {a,b}∗, |v| = 2}; (v) {w | #a(w) mod 3 = 0}. (20)

(c) (i) alternating 0s and 1s; (ii) over {a,b,c} at least one a and at least one b; (iii) every pair of adjacent 0s appears before any pair of adjacent 1s. (4 + 4 + 7)

(d) (i) decimal numbers divisible by 5 (no leading 0 except '0'); (ii) odd number of 0s or odd number of 1s; (iii) binary strings not containing 101. (4 + 3 + 3)

(e) At least one a and one c over {a,b,c}; no two consecutive 0s or no two consecutive 1s; even number of 1s; alternating 0s and 1s; strings whose last two symbols are NOT equal.

✓ Solution

```
(a) (i)   b∗(ab∗ab∗ab∗)∗
    (ii)  b∗(a ∪ ε)b∗(a ∪ ε)b∗(a ∪ ε)b∗
    (iii) (b ∪ ab ∪ aab)∗aaa(b ∪ ba ∪ baa)∗        – the only aaa block, other a-blocks have length
≤ 2
(b) (i)   union of the 6 orders:  Σ∗aΣ∗bΣ∗cΣ∗  ∪  Σ∗aΣ∗cΣ∗bΣ∗  ∪  Σ∗bΣ∗aΣ∗cΣ∗  ∪  Σ∗bΣ∗cΣ∗aΣ∗  ∪
Σ∗cΣ∗aΣ∗bΣ∗  ∪  Σ∗cΣ∗bΣ∗aΣ∗
    (ii)  aaaa∗b(bb)∗
    (iii) (aa)∗a(bb)∗  ∪  (aa)∗b(bb)∗        – exactly one of n, m is odd
    (iv)  aa(a ∪ b)∗aa  ∪  ab(a ∪ b)∗ab  ∪  ba(a ∪ b)∗ba  ∪  bb(a ∪ b)∗bb
    (v)   b∗(ab∗ab∗ab∗)∗
(c) (i)   (1 ∪ ε)(01)∗(0 ∪ ε)
    (ii)  Σ∗aΣ∗bΣ∗  ∪  Σ∗bΣ∗aΣ∗               (Σ = a ∪ b ∪ c)
    (iii) (0 ∪ 10)∗(1 ∪ ε)(1 ∪ 01)∗(0 ∪ ε)        – a part without 11, then a part without 00
(d) (i)   0  ∪  5  ∪  [1-9][0-9]∗(0 ∪ 5)
    (ii)  1∗0(1∗01∗0)∗1∗  ∪  0∗1(0∗10∗1)∗0∗
    (iii) 0∗(1+00+)∗(ε ∪ 1+ ∪ 1+0)             – after a block of 1s, at least two 0s unless the
string ends
(e)       Σ∗aΣ∗cΣ∗  ∪  Σ∗cΣ∗aΣ∗  ;   (1 ∪ 01)∗(0 ∪ ε)  ∪  (0 ∪ 10)∗(1 ∪ ε)  ;   0∗(10∗10∗)∗  ;   (1
∪ ε)(01)∗(0 ∪ ε)
          ε  ∪  0  ∪  1  ∪  (0 ∪ 1)∗(01 ∪ 10)
```

Each line was checked against its definition on all strings up to length 10 (length 6 over {a, b, c}, length 3 for decimal numbers).

Practice — Lesson 12 — describing, simplifying, choosing

(f) Actual — 2006-07 Q4(b), 2007-08 Q2(b), Jan-2020 Q4(a). (g) Actual — 2019-20 Q2(b). (h) Actual — Jan-2020 Q3(a). (i) Actual — 2021-22 Q3(d). (j) Actual — 2022-23 Q4(c). (k) Actual — 2023-24 Q4(b). (l) Actual — 2009-10 Q4. (m) Actual — 2013-14 Q5(b). (n) Actual — 2015-16 Q3(a).

(f) Describe: 0∗10∗; Σ∗1Σ∗; 1∗(01⁺)∗; (ΣΣΣ)∗; 0Σ∗0 ∪ 1Σ∗1 ∪ 0 ∪ 1; 01 ∪ 10; (0 ∪ ε)1∗; (01 ∪ 010)∗; (0 ∪ 1)∗; (0Σ∗) ∪ (Σ∗1); (1 + ε)(00∗1)∗0∗.

(g) Rewrite more simply: (i) ∅∗ ∪ a∗ ∪ b∗ ∪ (a ∪ b)∗; (ii) ((a∗b∗)∗(b∗a∗)∗)∗; (iii) (a∗b)∗ ∪ (b∗a)∗. (15)

(h) Evaluate: ∅∗∅∗∅∗∅∗; ∅∗ ∪ {ε}; (∅∗ ∪ 1){0,1}; (11)∗∅(00)∗; (∅∗ ∪ 0){0,1,11}. (10)

(i) Strings having two consecutive 0s as well as two consecutive 1s: A. (0+1)∗0011(0+1)∗ + (0+1)∗1100(0+1)∗; B. (0+1)∗(00(0+1)∗11 + 11(0+1)∗00)(0+1)∗; C. (0+1)∗00(0+1)∗ + (0+1)∗11(0+1)∗; D. 00(0+1)∗11 + 11(0+1)∗00. (12)

(j) DFA q0 (start) loops on 0, q0 −1→ q1 (accepting), q1 loops on 1, q1 −0→ q0. Options: A. 0∗1(1 + 00∗1)∗; B. 0∗1∗1 + 11∗0∗1; C. (0 + 1)∗1. (10)

(k) At most one occurrence of aa: A. (b+ab)∗(ε+a+aa)(b+ab)∗; B. (b+ab)∗(ε+a+aa)(b+ba)∗; C. (b+ab)∗(ε+a)(b+ba)∗; D. (b+ba)∗(ε+a+aa)(b+ba)∗. (12)

(l) L1 = 1(1 ∪ 0)∗000, L2 = (1(01)∗0)∗. Are 1000, 1000101010, 10000111000100000 in L1 ∘ L2? (10)

(m) Counter-example to (R1 ∪ R2)∗ = R1∗ ∪ R2∗.

(n) DFA and RE for: begins with 1 and ends with 0; length ≥ 3 with third symbol 0; even number of 0s or exactly two 1s; any string except 11 and 111; at least two 0s and at most one 1. (18)

✓ Solution

RE	Language
0∗10∗	exactly one 1
Σ∗1Σ∗	at least one 1
1∗(01⁺)∗	every 0 is followed by at least one 1 (no 00, does not end in 0)
(ΣΣΣ)∗	length a multiple of 3
0Σ∗0 ∪ 1Σ∗1 ∪ 0 ∪ 1	non-empty, starts and ends with the same symbol
01 ∪ 10	{01, 10}
(0 ∪ ε)1∗	0 optionally, then any 1s: {ε, 0, 1, 01, 11, 011, …}
(01 ∪ 010)∗	concatenations of blocks 01 and 010
(0 ∪ 1)∗	all binary strings
(0Σ∗) ∪ (Σ∗1)	starts with 0 or ends with 1
(1 + ε)(00∗1)∗0∗	no two consecutive 1s (checked)

(g) (i) (a ∪ b)∗ — it contains every other term (∅∗ = {ε}, a∗, b∗ ⊆ (a ∪ b)∗). (ii) (a ∪ b)∗ — a∗b∗ contains a and b, so (a∗b∗)∗ is already everything; the rest cannot add more. (iii) (a ∪ b)∗ — (a∗b)∗ = ε plus strings ending in b, (b∗a)∗ = ε plus strings ending in a; together every string.

(h) ∅∗∅∗∅∗∅∗ = {ε}{ε}{ε}{ε} = {ε}; ∅∗ ∪ {ε} = {ε}; (∅∗ ∪ 1){0,1} = {ε, 1}{0, 1} = {0, 1, 10, 11}; (11)∗∅(00)∗ = ∅ (a ∅ factor kills a concatenation); (∅∗ ∪ 0){0,1,11} = {ε, 0}{0, 1, 11} = {0, 1, 11, 00, 01, 011}.

(i) B only. A misses 001011 (00 and 11 not adjacent as 0011/1100); C accepts 00 (needs only one of the pairs); D misses 00110 (must start and end with the pairs). B: some 00 then later some 11, or 11 then later 00 — exactly 'contains both'.

(j) The DFA accepts exactly the strings ending in 1. **A and C** are correct. A: 0∗1 reaches q1, then each further piece is 1 (stay) or 00∗1 (go to q0 and back) — always ends in 1. B is wrong: it misses 0101 (and 1011).

(k) B only. A accepts aaab (two overlapping aa in aaa) and misses aaba (after aa, the right part (b+ab)∗ cannot end with a). C misses aa itself. D accepts baaa and misses abaa. B: left part (b+ab)∗ has no aa and ends with b, middle has at most one aa, right part (b+ba)∗ starts with b — so no aa crosses a boundary.

(l) L2 strings are built from blocks 1(01)∗0 = 10, 1010, … so they never contain 00 and are empty or end in 10. 1000: L1 = 1000, L2 = ε ✓. 1000101010: 1000 ∘ 101010 (= 1·01·01·0) ✓. 10000111000100000 ends in 00000, so its L2 part must be ε; the whole string starts with 1 and ends with 000, so it is in L1 ✓. All three belong.

(m) R1 = a, R2 = b: ab ∈ (a ∪ b)∗ but ab ∉ a∗ ∪ b∗.

(n) REs (DFAs follow the Lesson 4 recipe; Sipser Ex. 1.6/1.18):

```
begins 1, ends 0          1(0 ∪ 1)∗0
3rd symbol 0               (0 ∪ 1)(0 ∪ 1)0(0 ∪ 1)∗
even 0s or exactly two 1s  (1∗01∗0)∗1∗  ∪  0∗10∗10∗
except 11 and 111          ε  ∪  0  ∪  1  ∪  00  ∪  01  ∪  10  ∪  0(0 ∪ 1)(0 ∪ 1)  ∪  10(0 ∪ 1)  ∪
110  ∪  (0 ∪ 1)(0 ∪ 1)(0 ∪ 1)(0 ∪ 1)(0 ∪ 1)∗
≥ two 0s and ≤ one 1       000∗  ∪  1000∗  ∪  0100∗  ∪  000∗10∗
```

```
DFAs (sketch):  begins 1 ends 0: s –1→ A, s –0→ dead; A –0→ B*, A –1→ A; B –0→ B, B –1→ A
third symbol 0: s –Σ→ t –Σ→ u; u –0→ OK* (loops Σ), u –1→ dead
even 0s or exactly two 1s: product of parity-of-0s (2) and count-of-1s capped at 3 (4) → 8 states
except 11,111: count prefix 1s: ε*,1*,11,111 states; any 0 → ok*; 1111 → ok*
≥2 zeros, ≤1 one: (zeros 0/1/2+) × (ones 0/1) + dead on the second 1
```

Lesson 13 — RE → NFA (Thompson's Construction)

Slides 85–93 (Lec05)

Kleene's theorem: a language is regular iff some RE describes it (sl. 85–86). Lemma 1 (RE → NFA) is proved by structural induction on the RE: NFAs for the base cases a, ε, ∅, then plug the NFAs of the parts into the union, concatenation and star constructions of Lesson 10 (sl. 87–90).

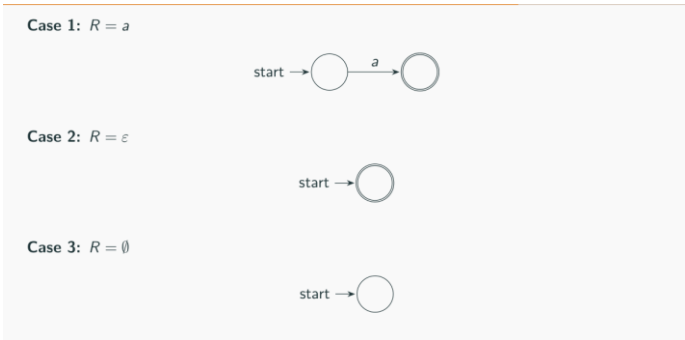


Figure 13.1 — Base cases: a, ε, ∅ (sl. 89)

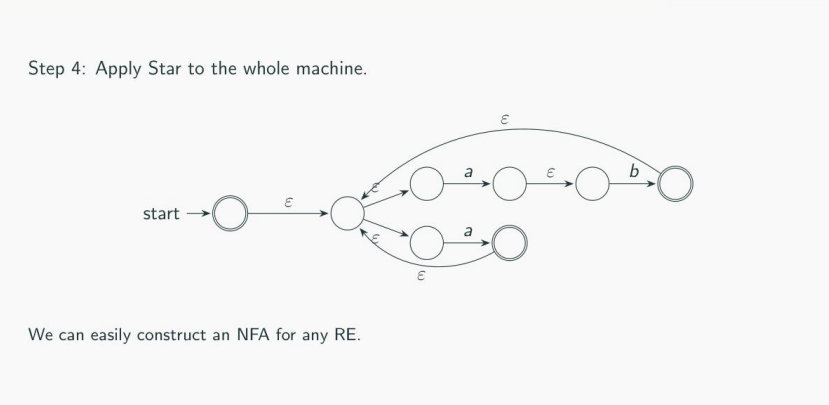


Figure 13.2 — (ab ∪ a)∗ built step by step (sl. 91–93)

Exam Approach — 'Using the Basic Principles'

1. Parse the RE (star binds tightest, then concatenation, then union).
2. Draw the tiny NFA for each symbol (2 states, one arrow).
3. Combine inside-out: concatenation = ε from accept of the first to start of the second; union = new start with two ε; star = new accepting start + ε back from accept states.
4. Do not 'optimise' unless allowed — the examiner checks the construction steps. Label the final start/accept.

Practice — Lesson 13

(a) Actual — 2019-20 Q3(a). (b) Actual — 2017-18 Q2(c). (c) Actual — 2008-09 Q7(c). (d) Actual — 2021-22 Q2(b).

(a) Construct the NFA for ((a∗)a)∗ using the basic principles. (5)

(b) Using the techniques of the closure proofs, draw the NFA for (a ∪ b)∗aba. (8)

(c) Convert (0 + 1)∗1(0 + 1) to an ε-NFA. (4⅓)

(d) Using the basic principles convert (((00)∗(11)) ∪ 01)∗ into an NFA. (10)

✓ Solution

Each answer below is the exact Thompson-style construction (one 2-state NFA per symbol, ε-glue for ∘, new start for ∪, new accepting start + back-ε for ∗). The edge lists were generated and tested against the RE on all strings up to length 8.

```
(a) ((a∗)a)∗: start s7, accept s8  (ε written as ε)
   s1 −a→ s2    s3 −ε→ s1    s2 −ε→ s1    s2 −ε→ s4    s3 −ε→ s4
   s5 −a→ s6    s4 −ε→ s5    s7 −ε→ s3    s6 −ε→ s3    s6 −ε→ s8
   s7 −ε→ s8
```

```
(b) (a ∪ b)∗aba: start s7, accept s14  (ε written as ε)
   s1 −a→ s2    s3 −b→ s4    s5 −ε→ s1    s5 −ε→ s3    s2 −ε→ s6
   s4 −ε→ s6    s7 −ε→ s5    s6 −ε→ s5    s6 −ε→ s8    s7 −ε→ s8
   s9 −a→ s10    s8 −ε→ s9    s11 −b→ s12    s10 −ε→ s11    s13 −a→ s14
   s12 −ε→ s13
```

```
(c) (0 + 1)∗1(0 + 1): start s7, accept s16  (ε written as ε)
   s1 −0→ s2    s3 −1→ s4    s5 −ε→ s1    s5 −ε→ s3    s2 −ε→ s6
   s4 −ε→ s6    s7 −ε→ s5    s6 −ε→ s5    s6 −ε→ s8    s7 −ε→ s8
   s9 −1→ s10    s8 −ε→ s9    s11 −0→ s12    s13 −1→ s14    s15 −ε→ s11
   s15 −ε→ s13    s12 −ε→ s16    s14 −ε→ s16    s10 −ε→ s15
```

```
(d) (((00)∗(11)) ∪ 01)∗: start s17, accept s18  (ε written as ε)
   s1 −0→ s2    s3 −0→ s4    s2 −ε→ s3    s5 −ε→ s1    s4 −ε→ s1
   s4 −ε→ s6    s5 −ε→ s6    s7 −1→ s8    s9 −1→ s10    s8 −ε→ s9
   s6 −ε→ s7    s11 −0→ s12    s13 −1→ s14    s12 −ε→ s13    s15 −ε→ s5
   s15 −ε→ s11    s10 −ε→ s16    s14 −ε→ s16    s17 −ε→ s15    s16 −ε→ s15
   s16 −ε→ s18    s17 −ε→ s18
```

Reading (b): s1 −a→ s2, s3 −b→ s4, union s5 → s1/s3, star s7 (start, accepting) → s5, then ε to the chain for a, b, a. A compact equivalent (allowed only if the question does not insist on basic principles): q0 loops on a, b; q0 −a→ q1 −b→ q2 −a→ q3.

Lesson 14 — DFA → RE (GNFA State Ripping) and Kleene's Theorem

Slides 94–111 (Lec05)

1. GNFA (sl. 94–99)

A generalised NFA has whole REs on its arrows: qi −R→ qj means 'read a block matching R'. Sipser's normal form: (1) the start state has arrows to every other state and none coming in; (2) a single accept state, arrows in from everywhere, none out; (3) start ≠ accept; (4) between all other states (including self-loops) exactly one arrow. Prepare a DFA: new start with ε to the old start; new accept with ε from old accepting states; missing arrows labelled ∅; parallel arrows merged with ∪ (sl. 97–99).

2. Ripping a State (sl. 100–103)

$$R_{new}(qi, qj) = R(qi, qj) \cup R(qi, q_{rip})\,(R(q_{rip}, q_{rip}))^{*}\,R(q_{rip}, qj)$$

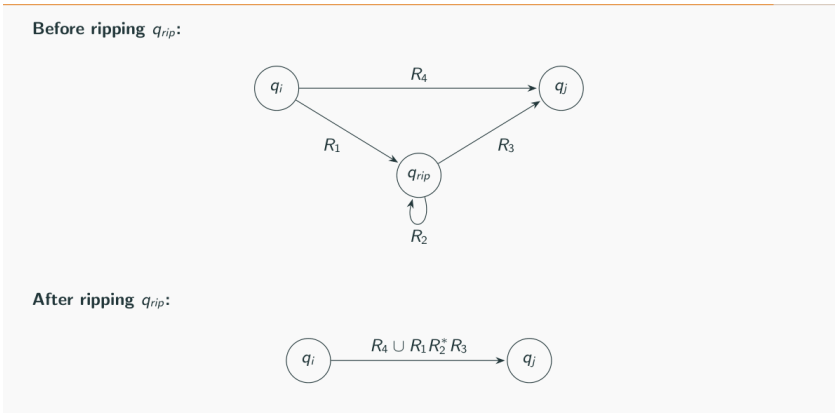


Figure 14.1 — Before and after ripping q_rip (sl. 102)

Algorithm Convert(G): if G has 2 states return the label qstart → qaccept; otherwise rip any internal state, repair every remaining pair (qi, qj) with the formula, and recurse (sl. 103). The result is correct but not necessarily the prettiest RE (sl. 104).

3. Worked Example: Binary Multiples of 3 (sl. 105–109)

```
Rip q2:  R(q1,q1) = ∅ ∪ 0 1* 0 = 01*0
Rip q1:  R(q0,q0) = 0 ∪ 1 (01*0)* 1
Rip q0:  R(qs,qa) = ε (0 ∪ 1(01*0)*1)* ε  =  (0 ∪ 1(01*0)*1)*
Sanity: 11 = 3 and 1001 = 9 match 1(…)*1.
```

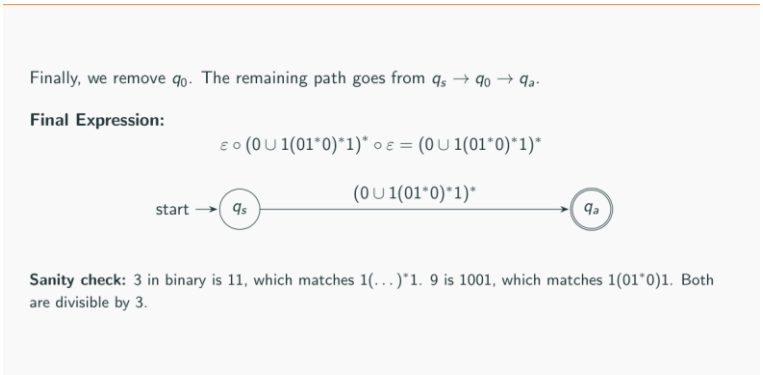


Figure 14.2 — Final GNFA (sl. 109)

4. Kleene's Theorem — Proof Answers (2012-13 Q8(b), 2008-09 Q7(a))

EXAM-READY ANSWER Theorem: L is regular ⇔ L = L(R) for some RE R. (⇐) Structural induction on R (Lesson 13): base cases a, ε, ∅ have NFAs; if R1, R2 have NFAs then so do R1 ∪ R2, R1R2, R1∗ by the closure constructions; NFAs recognise regular languages. (⇒) Let L = L(A) for a DFA A. Convert A into a GNFA (new start, single new accept, ∅ for missing arrows, ∪ for parallel ones). Rip internal states one at a time with R_new(qi,qj) = R(qi,qj) ∪ R(qi,q_rip)R(q_rip,q_rip)∗R(q_rip,qj). Claim: ripping keeps the language — any computation of the old GNFA that passes through q_rip enters it from some qi, loops some times and leaves to some qj, which the new label qi → qj matches; conversely every string matched by a new label is read by a path of the old GNFA. By induction on the number of states, when 2 states remain the single label R satisfies L(R) = L(A).

Exam Approach — DFA → RE

1. Add qs −ε→ start and accepting −ε→ qa. Merge parallel arrows with ∪.
2. Rip states with few arrows first (or follow the order the question gives).
3. For each ripped state list every pair (in-arrow, out-arrow) and write the new label; keep old labels with ∪.
4. Simplify only with safe identities (εR = R, R ∪ ∅ = R). Check with 2 strings.

Practice — Lesson 14

(a) Actual — 2013-14 Q6(c). (b) Actual — 2012-13 Q7(b). (c) Actual — 2011-12 Q7(c).

(a) Construct a regular expression from the NFA: q0 (start, accepting) −a→ q1; q1 −b→ q0; q1 −a→ q2; q2 loops on a; q2 −b→ q0. (8)

(b) Convert the DFA to an RE: 1 (start) −a→ 2, 2 −a→ 1, 2 loops on b, 1 −b→ 3, 3 −b→ 1, 3 −a→ 2; accepting 2 and 3. (7⅓)

(c) Derive the RE for the automaton: 1 (start) −a,b→ 2, 2 loops on a, 2 −b→ 3, 3 −a→ 1, 3 −b→ 2; all of 1, 2, 3 accepting. (7⅓)

✓ Solution

(a) Rip q2: R(q1, q0) = b ∪ a a∗ b. Rip q1: R(q0, q0) = a(b ∪ aa∗b) = aa∗b. Rip q0 (start and accepting, with qs, qa added): R(qs, qa) = (aa∗b)∗. In words: blocks of one or more a's each followed by a single b. (Program result ((a(b ∪ aa∗b))∗) — same language, checked.)

(b) Add qs −ε→ 1 and 2, 3 −ε→ qa. Rip 3: R(1,1) = bb, R(1,2) = a ∪ ba, R(1,qa) = b. Rip 2 (loop b): R(1,1) = bb ∪ (a ∪ ba)b∗a, R(1,qa) = b ∪ (a ∪ ba)b∗. Rip 1:

$$\mathbf{(bb \cup (a \cup ba)b*a)* (b \cup (a \cup ba)b*)}$$

Checked against the DFA on all strings up to length 9.

(c) All three states are accepting and every state has a move on both a and b (1: a,b; 2: a, b; 3: a, b), so the DFA is complete and accepts every string: **(a ∪ b)∗**. A blind GNFA run gives the longer ((a∪b)(a∪bb)∗ba)∗(ε ∪ (a∪b)(a∪bb)∗(ε∪b)) — the same language; look before you rip!

Quick Notes — Regular Languages

Revision points for Lessons 10–14.

L10 · Regular operations

- A ∪ B, A ∘ B, A∗ (k ≥ 0 pieces). ∅∗ = {ε}; L∅ = ∅; L{ε} = L.
- Union: new start, ε to both starts, F = F1 ∪ F2, δ(q0, a≠ε) = ∅.
- Concat: ε from F1 to q2, F = F2. Star: new accepting start, ε to q1, ε from F1 to q1.
- Old start made accepting = wrong (A = a∗b accepts a). Finite ∘ finite = finite.

L11 · Other closures

- Complement: complete DFA, flip F. NFA flip fails (q0 loop a + q0 −a→ q1).
- Product Q1 × Q2: AND → F1×F2, OR → either. A − B = A ∩ B̄. Reverse: reverse arrows → NFA.
- Subsets of regular languages need not be regular ({0^n1^n} ⊂ Σ∗).

L12 · REs

- Precedence ∗, ∘, ∪. R∅ = ∅, Rε = R, (R∗)∗ = R∗.
- Counts mod 3: b∗(ab∗ab∗ab∗)∗. ≤ k: (a|ε) pattern.
- Contains both X and Y: union of the two orders. Forbidden pattern: allowed blocks.
- Option questions: one counterexample per wrong option; watch overlaps (aaa has two aa).

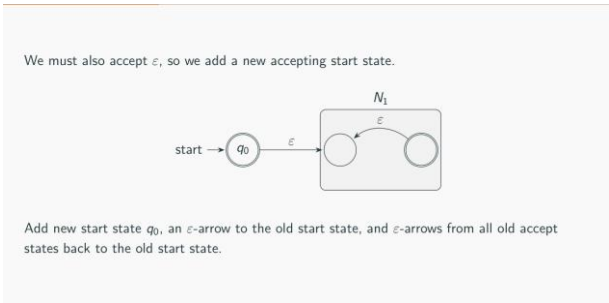
L13 · RE → NFA

- Base: a, ε, ∅. Inductive: plug into ∪, ∘, ∗ constructions (Thompson). Draw inside-out.

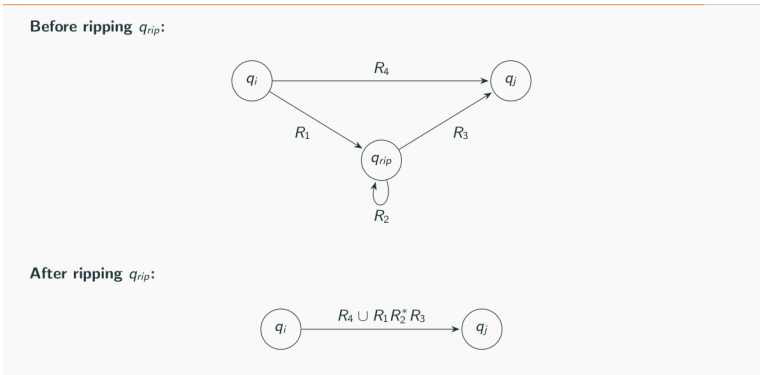
L14 · DFA → RE

- GNFA: new start/accept, ∅ for missing, ∪ for parallel arrows.
- R_new = R(i,j) ∪ R(i,rip)R(rip,rip)∗R(rip,j). Div-by-3: (0 ∪ 1(01∗0)∗1)∗.
- Kleene: RE ⇔ NFA ⇔ DFA.

Key Diagrams to Reproduce



Star construction (sl. 32) — Fig. 10.3



Ripping a state (sl. 102) — Fig. 14.1